JAIRAM PUBLIC SCHOOL (SENIOR SECONDARY)
SALEM.



ACADEMIC YEAR: 2026-2027

A PROJECT REPORT ON

"Text File Compressor"

SUBMITTED BY

ROLL NO	**:**	
NAME	**:**	**DHARUN L**
CLASS	**:**	**XII**
SUBJECT	**:**	**COMPUTER SCIENCE**
SUB CODE	**:**	**083**

PROJECT GUIDE: **MS.ARUNA A., MCA, M.Phil., B.Ed.**

PGT (COMPUTER SCIENCE)

DEPARTMENT OF COMPUTER SCIENCE

JAIRAM PUBLIC SCHOOL

SALEM.

JAIRAM PUBLIC SCHOOL SALEM



CERTIFICATE

This is to certify that Cadet (**DHARUN L)** Roll No: _____ has successfully completed the project work entitled **"Text File Compressor"** in the subject **Computer Science (083)** laid down in the regulations of **CBSE** for the purpose of **Practical Examination in Class XII** to be held in Jairam Public School on _____.

INTERNAL EXAMINER **PRINCIPAL** **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

Apart from the efforts for me, the success of my project depends largely on the encouragement and guidelines of many others. I take this opportunity to express my gratitude to the people who have been instrumental in the successful completion of this project.

I express deep sense of gratitude to almighty God for giving me strength for the successful completion of the project.

I express my heartfelt gratitude to my parents for constant encouragement while carrying out this project.

I express my sincere gratitude to our honorable Principal **Mr. I. PAUL FRANCIS XAVIER**, Jairam Public School who has been continuously motivating and extending their helping hand to us.

I express my sincere thanks to our **CO-ORDINATORS**, Jairam Public School Salem, for constant encouragement and the guidance provided during this project.

My sincere thanks to **Ms. ARUNA A., MCA., M.Phil., B.Ed.**, A guide, Mentor all the above a friend, who critically reviewed my project and helped in solving each and every problem, occurred during implementation of the project.

The guidance and support received from all the members who contributed and who are contributing to this project, was vital for the success of the project. I am grateful for their constant support and help.

CONTENTS

1	INTRODUCTION	5
2	SYSTEM REQUIREMENTS	9
3	SOURCE CODE	10
4	SAMPLE OUTPUT	26
5	CONCLUSION	31
6	BIBLIOGRAPHY	32

INTRODUCTION:

The Text Compressor is a desktop application developed using Python, designed to efficiently compress and decompress text files using a hybrid Word-Level and Character-Level Huffman Coding algorithm. It converts standard text files into a custom binary format with the extension *.vlr*. With the increasing need for efficient data storage, this project demonstrates how Data Structures and File Handling can be utilized to save disk space.

This application allows users to browse for files, compress them, view live statistics (such as space saved and compression ratio), and decompress them back to their original state. Furthermore, it integrates a MySQL database to act as a logbook, permanently storing the history and statistics of all compressed files. The main aim of the project is to help students understand the practical implementation of trees, priority queues, binary file handling, and database integration using Python.

The main objectives of the project are:

1. To develop a user-friendly GUI for file compression and decompression.
2. To implement a hybrid Huffman Coding algorithm (using word and character levels) to reduce file sizes.
3. To safely write and read binary files and manipulate bit strings.
4. To automatically calculate mathematical statistics like Space Saved and Compression Ratio.
5. To maintain a persistent record of all operations using a MySQL Database.
6. To introduce students to Python programming concepts such as Tkinter, File Handling (pickle), priority queues (heapq), and database connectivity (mysql.connector).

Tools and Technologies Used

1. Python Programming Language – for writing the application logic.
2. Tkinter – a standard Python library for creating the graphical user interface.
3. MySQL Database – for backend data storage of compression logs.
4. mysql-connector-python – to establish a connection between Python and the MySQL server.
5. heapq Module – to implement a Min-Heap (Priority Queue) for efficiently building the Huffman tree.
6. pickle Module – to serialize the frequency dictionary metadata and store it inside the compressed file.
7. re (Regex) Module – to accurately tokenize words, spaces, and punctuation from the text.

Description of the System

The Text Compressor System consists of four main modules:

1. Compression Module

- Tokenizes the input text into words and non-words.
- Calculates frequencies. Frequent words go to a word-level dictionary, while rare words are broken down into a character-level dictionary.
- Builds two Huffman Trees using the heapq module.
- Generates binary codes (0s and 1s) and encodes the text.
- Pads the binary string and writes it to a .vlr binary file alongside the serialized frequency metadata.

2. Decompression Module

- Reads the .vlr binary file and extracts the pickled metadata to rebuild the exact Huffman trees.
- Converts the binary bytes back into a string of 0s and 1s.
- Traverses the rebuilt tree bit-by-bit to accurately restore the original text.
- Saves the restored output as a .txt file.

3. Database Logging Module

- Automatically creates a database (compressor_db) and table (compression_logs) on the first run.
- Records the Original Size, Compressed Size, Space Saved, and Compression Ratio every time a file is compressed.

4. GUI Module

- Built completely using Python procedural programming and tkinter.
- Provides Entry boxes, "Browse" buttons, and a live statistics display panel.
- Features a separate window to view the MySQL database logs.

Working of the System

1. The user launches the application and is presented with the Tkinter GUI.
2. The user clicks "Browse" to select a standard .txt file.
3. Upon clicking "Compress to .vlr", the system tokenizes the file and builds the Huffman trees.
4. The text is encoded into binary, and a .vlr file is generated in the same directory.

5. The application calculates the compression ratio and space saved, displaying it in green text on the interface.
6. These statistics are simultaneously pushed to the MySQL Database via SQL INSERT commands.
7. The user can click "View Database Logs" to open a secondary window fetching the history via a SELECT query.
8. To decompress, the user browses for a .vlr file and clicks "Decompress", which reverses the binary logic to restore the exact original text.

Features

1. Custom .vlr file extension for compressed outputs.
2. Advanced Hybrid compression (Word + Character) targeting higher compression ratios.
3. Seamless automated MySQL database setup and record keeping.
4. Live file size statistics displayed dynamically on the interface.
5. Simple and interactive Graphical User Interface (GUI).
6. Complete error handling (e.g., alert boxes if the wrong file type is selected).

Applications and Benefits

- Can be used as a personal desktop utility to save disk space for large text-based logs or documents.
- Helps students learn real-world algorithms like Huffman Coding.
- Demonstrates the connection between frontend (GUI), backend logic (Python), and database (MySQL).
- Automates data logging, removing the need for manual record-keeping.

SYSTEM REQUIREMENTS

2.1. Hardware Requirements

- **Processor:** Intel Core i3 or above (any modern CPU)
- **RAM:** Minimum 4 GB (8 GB recommended for handling large text files)
- **Storage:** At least 500 MB free space for Python, libraries, XAMPP/MySQL, and project files
- **Display:** Standard monitor with 1024×768 resolution or higher

2.2. Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **Database Server:** XAMPP, WAMP, or standalone MySQL Server
- **Python Version:** Python 3.8 or above
- **Libraries/Modules:**
  - tkinter (for GUI widgets and file dialogs)
  - mysql-connector-python (for database operations)
  - os (for file path operations and calculating byte sizes)
  - heapq (for priority queue and tree building)
  - pickle (for metadata serialization)
  - re (for regular expressions and tokenization)

2.3. Additional Requirements

- **Text Editor/IDE:** IDLE, PyCharm, VS Code, or any Python IDE.
- **MySQL Setup:** The MySQL server must be actively running (e.g., Apache and MySQL started in XAMPP control panel) before running the program.

SOURCE CODE:

```
import os
import heapq
import pickle
import re
import tkinter as tk
from tkinter import filedialog, messagebox
import mysql.connector

# --- GLOBAL CONSTANTS & DB CONFIGURATION ---
PLACEHOLDER = "__PLACEHOLDER__"
END_WORD = "__END_WORD__"

DB_HOST = "localhost"
DB_USER = "root"
DB_PASS = "123456"

global_node_counter = 0

# ==========================================================
# 1. DATABASE LOGIC (MYSQL CONNECTIVITY)
# ==========================================================

def setup_database():
```

```
    try:
        conn = mysql.connector.connect(host=DB_HOST, user=DB_USER,
password=DB_PASS, auth_plugin="mysql_native_password")
        cursor = conn.cursor()
        cursor.execute("CREATE DATABASE IF NOT EXISTS compressor_db")
        cursor.execute("USE compressor_db")
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS compression_logs (
                log_id INT AUTO_INCREMENT PRIMARY KEY,
                file_name VARCHAR(100),
                original_size INT,
                compressed_size INT,
                space_saved INT,
                compression_ratio DECIMAL(5,2)
            )
        """)
        conn.commit()
        conn.close()
    except Exception as e:
        print("MySQL Connection Error:", e)


def log_to_database(filename, orig_size, comp_size, space_saved, ratio):
    try:
        conn = mysql.connector.connect(host=DB_HOST, user=DB_USER,
password=DB_PASS, database="compressor_db",
auth_plugin="mysql_native_password")
```

```
        cursor = conn.cursor()
        query = """INSERT INTO compression_logs
                (file_name, original_size, compressed_size, space_saved,
compression_ratio)
                VALUES (%s, %s, %s, %s, %s)"""
        values = (filename, orig_size, comp_size, space_saved, ratio)
        cursor.execute(query, values)
        conn.commit()
        conn.close()
    except Exception as e:
        print("Failed to log data to MySQL:", e)

# ============================================================
# 2. CORE COMPRESSION LOGIC (HUFFMAN)
# ============================================================

def tokenize(text):
    tokens = re.findall(r'\w+|\W', text)
    return tokens

def build_frequency_dicts(tokens):
    temp_freq = {}
    for token in tokens:
        if token in temp_freq:
            temp_freq[token] += 1
        else:
```

```
            temp_freq[token] = 1

    word_freq = {PLACEHOLDER: 0}
    char_freq = {END_WORD: 1}

    for token, count in temp_freq.items():
        if not token.isalnum() or count > 2:
            word_freq[token] = count
        else:
            word_freq[PLACEHOLDER] += count
            for char in token:
                if char in char_freq:
                    char_freq[char] += count
                else:
                    char_freq[char] = count
    return word_freq, char_freq

def build_huffman_tree(freq_dict):
    global global_node_counter
    heap = []
    for key, freq in freq_dict.items():
        if freq > 0:
            global_node_counter += 1
            heapq.heappush(heap, [freq, global_node_counter, key, None, None])
```

```
    if len(heap) == 0:
        return None

    if len(heap) == 1:
        node = heapq.heappop(heap)
        global_node_counter += 1
        root = [node[0], global_node_counter, None, node, None]
        return root

    while len(heap) > 1:
        left_node = heapq.heappop(heap)
        right_node = heapq.heappop(heap)
        merged_freq = left_node[0] + right_node[0]
        global_node_counter += 1
        merged_node = [merged_freq, global_node_counter, None, left_node,
right_node]
        heapq.heappush(heap, merged_node)

    return heapq.heappop(heap)

def generate_codes_helper(node, current_code, codes_dict, reverse_codes_dict):
    if node is None:
        return
    token = node[2]
    left_child = node[3]
    right_child = node[4]
```

```
    if token is not None:
        codes_dict[token] = current_code
        reverse_codes_dict[current_code] = token
        return

    generate_codes_helper(left_child, current_code + "0", codes_dict,
reverse_codes_dict)
    generate_codes_helper(right_child, current_code + "1", codes_dict,
reverse_codes_dict)

def generate_codes(word_freq, char_freq):
    word_root = build_huffman_tree(word_freq)
    char_root = build_huffman_tree(char_freq)

    word_codes, reverse_word_codes = {}, {}
    char_codes, reverse_char_codes = {}, {}

    generate_codes_helper(word_root, "", word_codes, reverse_word_codes)
    generate_codes_helper(char_root, "", char_codes, reverse_char_codes)

    return word_codes, reverse_word_codes, char_codes, reverse_char_codes

def pad_encoded_text(encoded_text):
    extra_padding = 8 - (len(encoded_text) % 8)
    for i in range(extra_padding):
```

```
        encoded_text += "0"
    padded_info = "{0:08b}".format(extra_padding)
    return padded_info + encoded_text

def get_byte_array(padded_encoded_text):
    b = bytearray()
    for i in range(0, len(padded_encoded_text), 8):
        byte = padded_encoded_text[i:i+8]
        b.append(int(byte, 2))
    return b

def remove_padding(padded_encoded_text):
    padded_info = padded_encoded_text[:8]
    extra_padding = int(padded_info, 2)
    padded_encoded_text = padded_encoded_text[8:]
    return padded_encoded_text[:-extra_padding]

# ============================================================
# 3. MAIN COMPRESS & DECOMPRESS PROCESSES
# ============================================================

def execute_compression(input_path, output_path):
    with open(input_path, 'r', encoding='utf-8') as file:
        text = file.read()
```

```
tokens = tokenize(text)
word_freq, char_freq = build_frequency_dicts(tokens)
word_codes, _, char_codes, _ = generate_codes(word_freq, char_freq)

encoded_text = ""
for token in tokens:
    if token in word_codes and token != PLACEHOLDER:
        encoded_text += word_codes[token]
    else:
        encoded_text += word_codes[PLACEHOLDER]
        for char in token:
            encoded_text += char_codes[char]
        encoded_text += char_codes[END_WORD]

padded_encoded_text = pad_encoded_text(encoded_text)
byte_array = get_byte_array(padded_encoded_text)

metadata = {'word_freq': word_freq, 'char_freq': char_freq}
pickled_metadata = pickle.dumps(metadata)
meta_length = len(pickled_metadata)

with open(output_path, 'wb') as output:
    output.write(meta_length.to_bytes(4, byteorder='big'))
    output.write(pickled_metadata)
    output.write(byte_array)
```

```
def execute_decompression(input_path, output_path):
    with open(input_path, 'rb') as file:
        meta_length = int.from_bytes(file.read(4), byteorder='big')
        pickled_metadata = file.read(meta_length)
        metadata = pickle.loads(pickled_metadata)

        bit_string = ""
        byte = file.read(1)
        while len(byte) > 0:
            byte = ord(byte)
            bits = bin(byte)[2:].rjust(8, '0')
            bit_string += bits
            byte = file.read(1)

    word_freq = metadata['word_freq']
    char_freq = metadata['char_freq']
    _, reverse_word_codes, _, reverse_char_codes = generate_codes(word_freq,
char_freq)

    encoded_text = remove_padding(bit_string)

    decoded_text = []
    current_code = ""
    is_parsing_chars = False
```

```
    for bit in encoded_text:
        current_code += bit
        if not is_parsing_chars:
            if current_code in reverse_word_codes:
                token = reverse_word_codes[current_code]
                if token == PLACEHOLDER:
                    is_parsing_chars = True
                else:
                    decoded_text.append(token)
                current_code = ""
        else:
            if current_code in reverse_char_codes:
                char = reverse_char_codes[current_code]
                if char == END_WORD:
                    is_parsing_chars = False
                else:
                    decoded_text.append(char)
                current_code = ""

    with open(output_path, 'w', encoding='utf-8') as output:
        output.write("".join(decoded_text))

# ==========================================================
# 4. GUI LOGIC (PROCEDURAL TKINTER)
# ==========================================================
```

```
def browse_compress_file():
    path = filedialog.askopenfilename(title="Select Text File", filetypes=(("Text Files", "*.txt"), ("All Files", "*.*")))
    if path:
        compress_entry_var.set(path)

def browse_decompress_file():
    path = filedialog.askopenfilename(title="Select .vlr File", filetypes=(("VLR Compressed Files", "*.vlr"), ("All Files", "*.*")))
    if path:
        decompress_entry_var.set(path)

def display_statistics(original_size, compressed_size, process_type, filename=""):
    if process_type == "compress":
        space_saved = original_size - compressed_size
        ratio = (compressed_size / original_size) * 100 if original_size > 0 else 100.00
        stats_text = (
            f"--- Compression Statistics ---\n"
            f"Original Size: {original_size} bytes\n"
            f"Compressed Size: {compressed_size} bytes\n"
            f"Space Saved: {space_saved} bytes\n"
            f"Compression Ratio: {ratio:.2f}% of original size"
        )
        stats_label.config(text=stats_text, fg="#1B5E20")
```

```
        log_to_database(filename, original_size, compressed_size, space_saved,
ratio)
    else:
        stats_text = f"--- Decompression Statistics ---\nProcess Successful!\nRestored
Size: {original_size} bytes"
        stats_label.config(text=stats_text, fg="#1B5E20")


def ui_compress_action():
    input_path = compress_entry_var.get()
    if not input_path or not os.path.exists(input_path):
        messagebox.showerror("Error", "Please select a valid file.")
        return
    output_path = input_path.rsplit('.', 1)[0] + ".vlr"
    try:
        stats_label.config(text="Status: Compressing... Please wait.", fg="blue")
        root.update()
        execute_compression(input_path, output_path)
        original_size = os.path.getsize(input_path)
        compressed_size = os.path.getsize(output_path)
        filename = os.path.basename(input_path)
        display_statistics(original_size, compressed_size, "compress", filename)
        messagebox.showinfo("Success", f"File compressed successfully!\nSaved as:
{os.path.basename(output_path)}")
    except Exception as e:
        messagebox.showerror("Error", f"An error occurred:\n{str(e)}")
```

```
def ui_decompress_action():
    input_path = decompress_entry_var.get()
    if not input_path or not os.path.exists(input_path):
        messagebox.showerror("Error", "Please select a valid .vlr file.")
        return
    output_path = input_path.replace(".vlr", "_decompressed.txt")
    try:
        stats_label.config(text="Status: Decompressing... Please wait.", fg="blue")
        root.update()
        execute_decompression(input_path, output_path)
        decompressed_size = os.path.getsize(output_path)
        display_statistics(decompressed_size, 0, "decompress")
        messagebox.showinfo("Success", f"File decompressed successfully!\nSaved
as: {os.path.basename(output_path)}")
    except Exception as e:
        messagebox.showerror("Error", f"An error occurred:\n{str(e)}")


def open_database_logs():
    log_window = tk.Toplevel(root)
    log_window.title("MySQL Compression Logs")
    log_window.geometry("650x300")
    text_area = tk.Text(log_window, wrap="none", font=("Courier", 10))
    text_area.pack(expand=True, fill="both", padx=10, pady=10)
    try:
```

```
        conn = mysql.connector.connect(host=DB_HOST, user=DB_USER,
password=DB_PASS, database="compressor_db",
auth_plugin="mysql_native_password")
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM compression_logs ORDER BY log_id
DESC")
        rows = cursor.fetchall()
        if not rows:
            text_area.insert("end", "No records found.\n")
        else:
            header = f"{'ID':<5} | {'File Name':<20} | {'Orig(B)':<10} |
{'Comp(B)':<10} | {'Saved(B)':<10} | {'Ratio(%)':<8}\n"
            text_area.insert("end", header + "-"*85 + "\n")
            for row in rows:
                fname = (row[1][:17] + "...") if len(row[1]) > 20 else row[1]
                line = f"{row[0]:<5} | {fname:<20} | {row[2]:<10} | {row[3]:<10} |
{row[4]:<10} | {row[5]:<8}\n"
                text_area.insert("end", line)
        conn.close()
    except Exception as e:
        text_area.insert("end", f"Database Error: {e}")
    text_area.config(state="disabled")


# ==========================================================
# 5. GUI INITIALIZATION
# ==========================================================
```

```
if __name__ == "__main__":
    setup_database()

    root = tk.Tk()
    root.title("Text Compressor (.vlr Format)")
    root.geometry("600x600")
    root.configure(bg="#f4f4f4")

    compress_entry_var = tk.StringVar()
    decompress_entry_var = tk.StringVar()

    tk.Label(root, text="Text Compressor", font=("Helvetica", 18, "bold"),
bg="#f4f4f4").pack(pady=(20, 5))
    tk.Label(root, text="Word & Character Level Huffman Coding",
font=("Helvetica", 10, "italic"), bg="#f4f4f4").pack(pady=(0, 20))

    comp_frame = tk.LabelFrame(root, text=" Compress File ", bg="#f4f4f4",
padx=10, pady=10)
    comp_frame.pack(fill="x", padx=20, pady=5)
    tk.Entry(comp_frame, textvariable=compress_entry_var, width=50).grid(row=0,
column=0, padx=5, pady=5)
    tk.Button(comp_frame, text="Browse", width=10,
command=browse_compress_file).grid(row=0, column=1, padx=5, pady=5)
    tk.Button(comp_frame, text="Compress to .vlr", width=20, bg="#4CAF50",
fg="white", command=ui_compress_action).grid(row=1, column=0,
columnspan=2, pady=5)
```

```
    decomp_frame = tk.LabelFrame(root, text=" Decompress .vlr File ",
bg="#f4f4f4", padx=10, pady=10)

    decomp_frame.pack(fill="x", padx=20, pady=15)

    tk.Entry(decomp_frame, textvariable=decompress_entry_var,
width=50).grid(row=0, column=0, padx=5, pady=5)

    tk.Button(decomp_frame, text="Browse", width=10,
command=browse_decompress_file).grid(row=0, column=1, padx=5, pady=5)

    tk.Button(decomp_frame, text="Decompress File", width=20, bg="#2196F3",
fg="white", command=ui_decompress_action).grid(row=1, column=0,
columnspan=2, pady=5)


    stats_frame = tk.Frame(root, bg="#f4f4f4")

    stats_frame.pack(fill="both", expand=True, padx=20, pady=5)

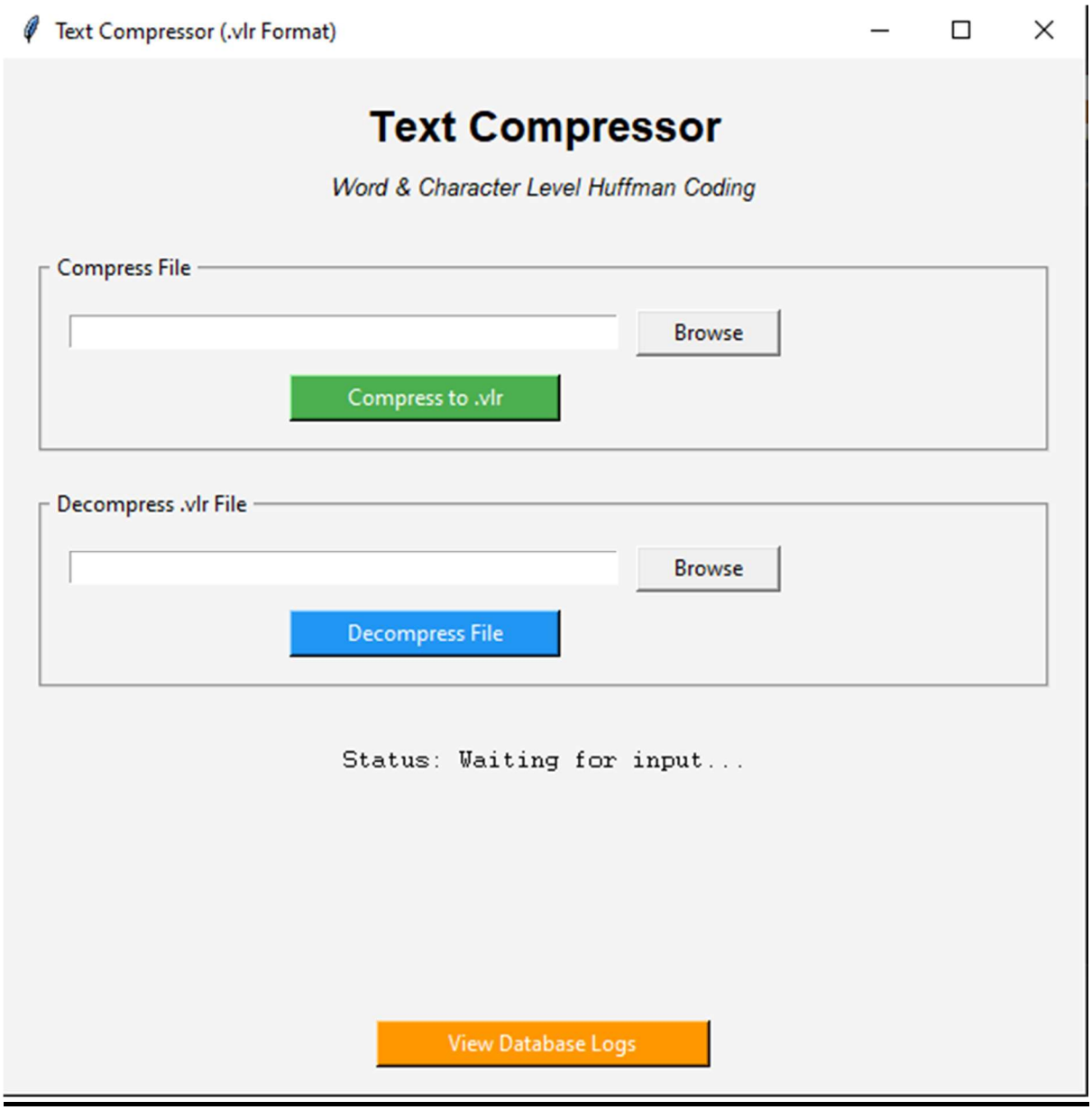

    stats_label = tk.Label(stats_frame, text="Status: Waiting for input...",
font=("Courier", 11), bg="#f4f4f4")

    stats_label.pack(pady=10)

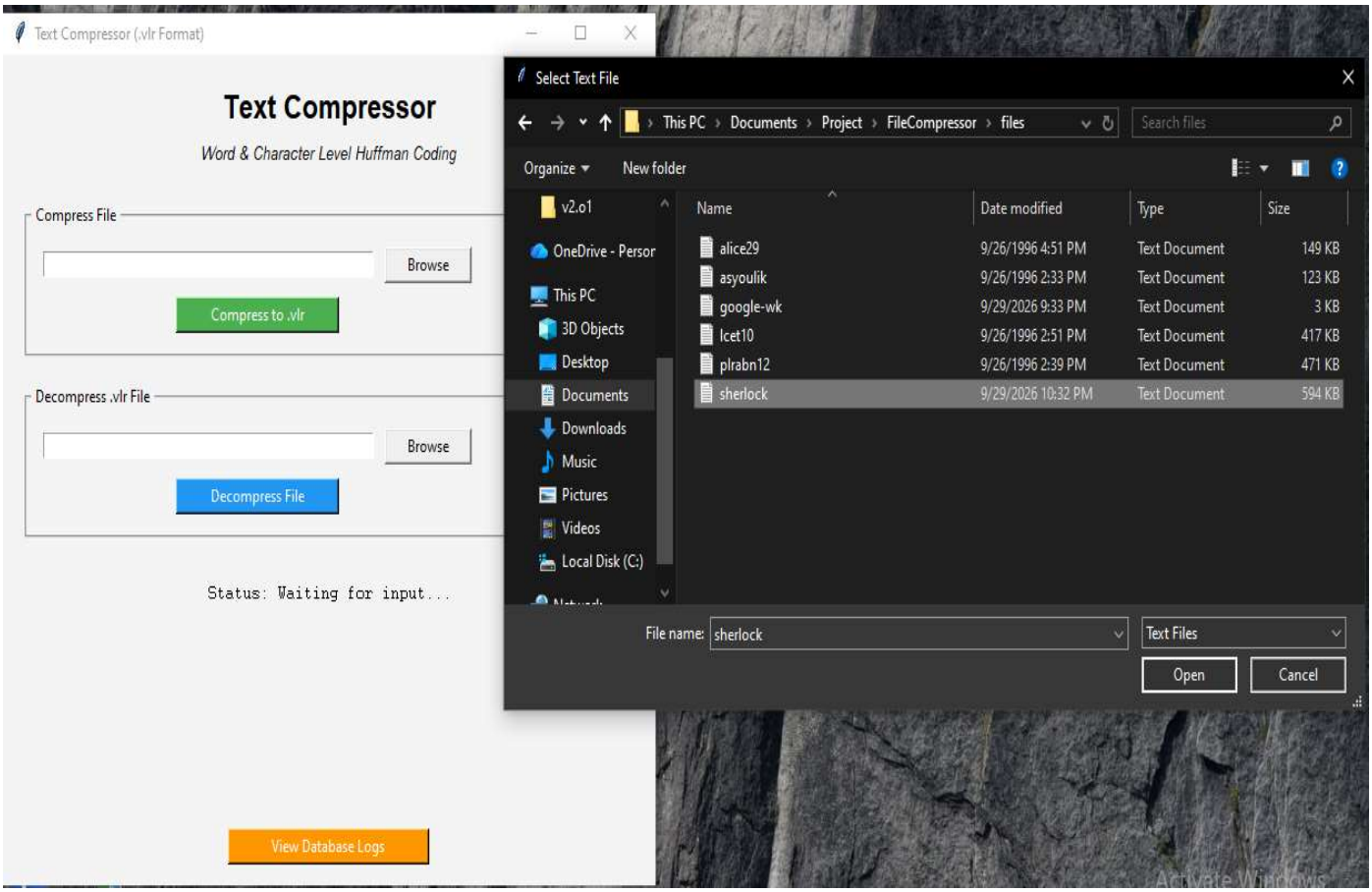

    tk.Button(stats_frame, text="View Database Logs", width=25, bg="#FF9800",
fg="white", command=open_database_logs).pack(side="bottom", pady=10)


    root.mainloop()
```

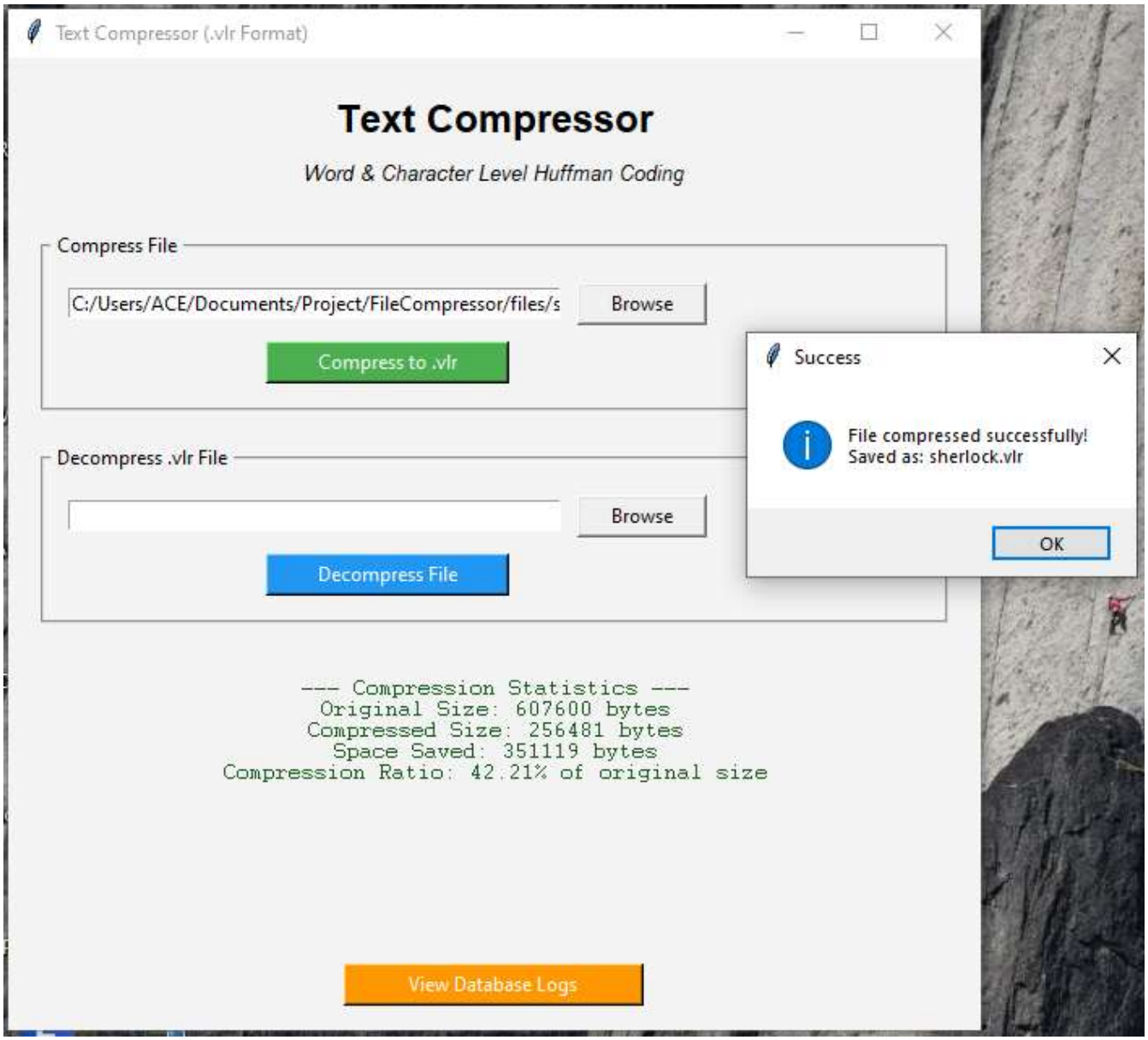
SAMPLE OUTPUT:



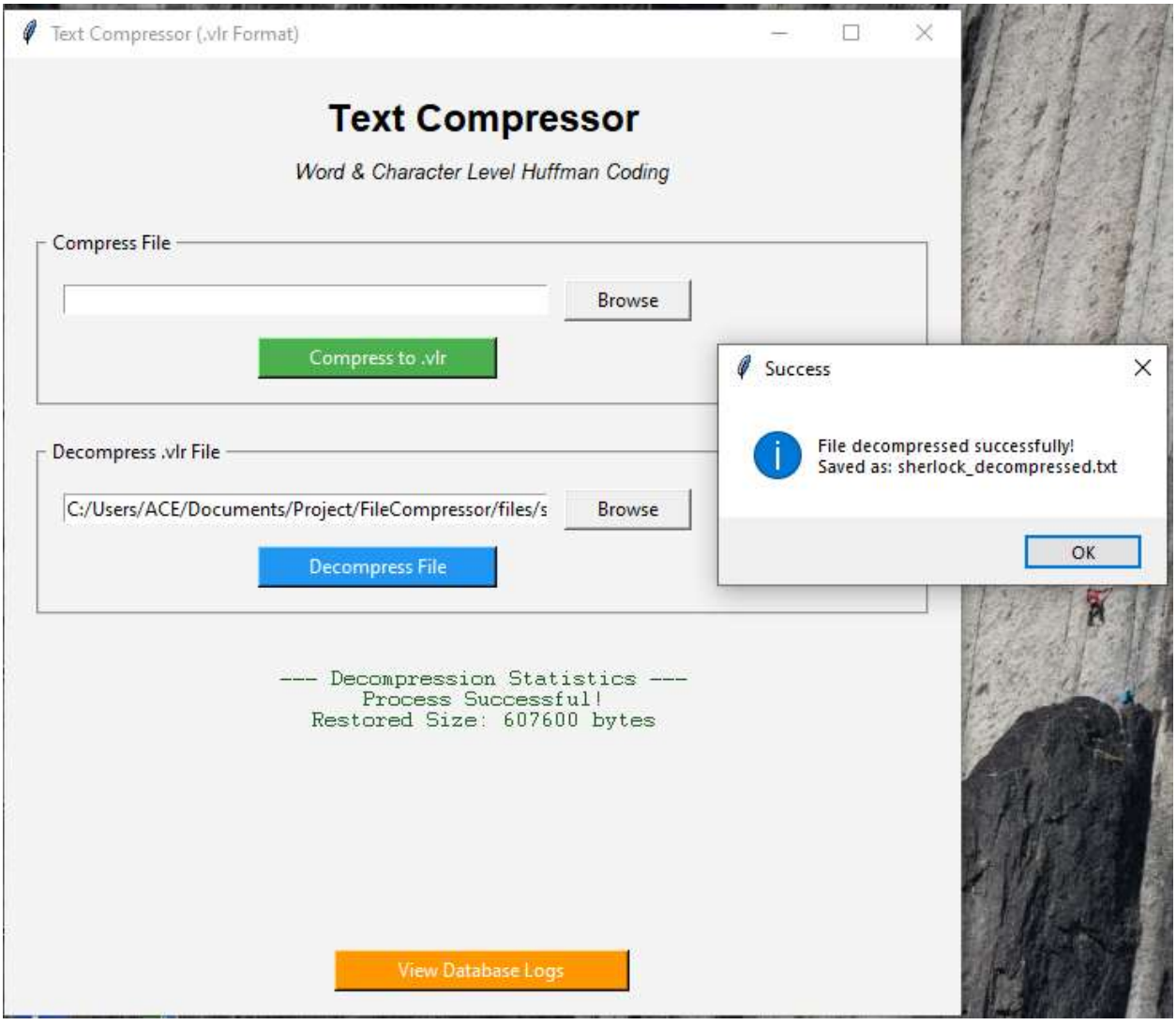
SELECTING A FILE FOR COMPRESSION:



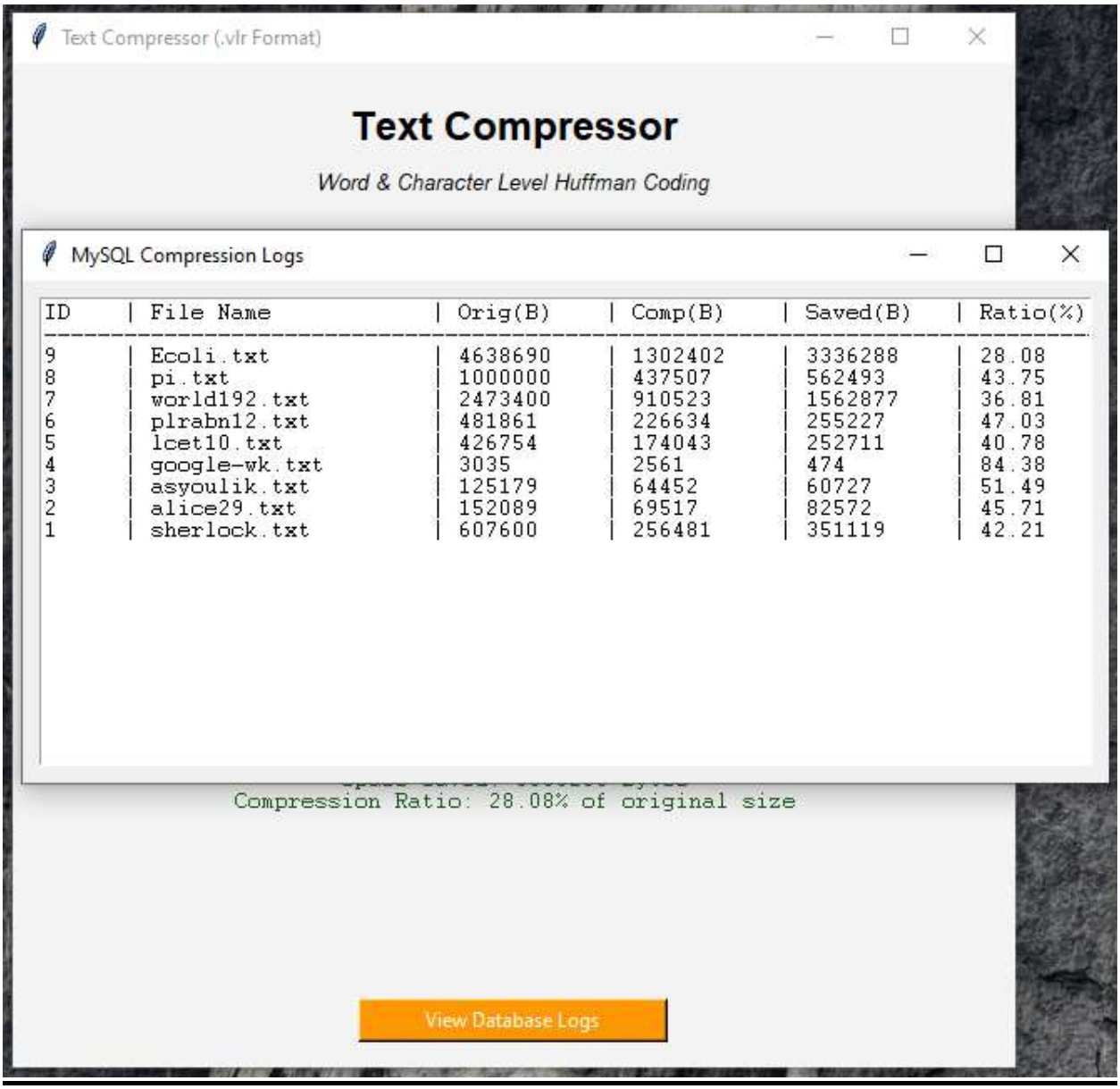
COMPRESSING THE FILE:



DECOMPRESSING THE FILE:



MYSQL LOGS:


Text Compressor (.vlr Format)
Text Compressor
Word & Character Level Huffman Coding
MySQL Compression Logs
ID | File Name | Orig(B) | Comp(B) | Saved(B) | Ratio(%)
9 | Ecoli.txt | 4638690 | 1302402 | 3336288 | 28.08
8 | pi.txt | 1000000 | 437507 | 562493 | 43.75
7 | world192.txt | 2473400 | 910523 | 1562877 | 36.81
6 | plrabn12.txt | 481861 | 226634 | 255227 | 47.03
5 | lcet10.txt | 426754 | 174043 | 252711 | 40.78
4 | google-wk.txt | 3035 | 2561 | 474 | 84.38
3 | asyoulik.txt | 125179 | 64452 | 60727 | 51.49
2 | alice29.txt | 152089 | 69517 | 82572 | 45.71
1 | sherlock.txt | 607600 | 256481 | 351119 | 42.21
Compression Ratio: 28.08% of original size
View Database Logs

CONCLUSION:

The Text Compressor investigatory project has been a practical demonstration of how core computer science algorithms can be applied to real-life situations. By using Data Structures (Trees, Min-Heaps), Python File Handling (Binary mode, Object Serialization), and system connectivity via MySQL, the project successfully mimics a real-world compression utility like WinRAR or 7-Zip, albeit focused on textual data.

The project highlights important programming skills such as algorithmic thinking, error checking, and database management. Features like Word-Level and Character-Level hybrid processing make the compression logic highly efficient, while the MySQL integration showcases automated record-keeping. Through this work, I gained a much deeper understanding of how data is fundamentally handled at the bit and byte level, while strengthening my knowledge of Python's procedural programming capabilities.

In conclusion, the project met its objectives and provided a valuable learning experience. It shows the versatility of Python and its ability to integrate multiple modules (like tkinter, heapq, pickle, and mysql-connector) into a cohesive, user-friendly application. With further improvements, such as adding folder compression or integrating dictionary-based algorithms like LZ77, the system could evolve into a much more advanced tool. Overall, this investigatory project has been highly enriching and has laid a strong foundation for future studies in computer science.

BIBLIOGRAPHY:

- https://docs.python.org/3/library/heapq.html (For understanding Priority Queues and Heaps)
- https://docs.python.org/3/library/pickle.html (For Object Serialization techniques)
- https://www.geeksforgeeks.org/huffman-coding-greedy-algo-3/ (For logic regarding tree building and encoding strings)
- https://dev.mysql.com/doc/connector-python/en/ (For connecting Python Tkinter to the MySQL database)

REFERENCES:

- Computer Science with Python - Textbook for Class XII (NCERT)
- Computer Science with Python by Sumita Arora (Class XII)